

P2P File-Sharing Application using Go, Java, and Python

<https://github.com/dobaj/cisc468-final-project>

Alice Fraimovich (20356347, 21amf28@queensu.ca)

Matt Dobaj (20350312, 21mtd4@queensu.ca)

Nicholas Wang (20364023, 22nw@queensu.ca)

ABSTRACT

This report analyzes the structure and implementation of our peer-to-peer file-sharing application, including detailing the system overview and architecture, protocol design and communication flow, algorithms and encryption scheme utilized and message format and types. We outline the process of designing a secure file-sharing system that guarantees integrity, and explain our reasoning behind each of the design choices made. Finally, we discuss error handling and edge cases, any problems we had run into and libraries used. In the first section, we demonstrate our planning through outlining our objectives for our application, and any sources we used in our research for this project. Our next section delves into our precise architecture and overview of the system(s) we implemented. Lastly, we lay out our process for testing and implementing the final aspects of the file-sharing application, including error-handling and edge cases, along with detailing all libraries used.

1 INTRODUCTION

A P2P file-sharing application enables users to send and receive files directly between peers. For our application, these users must share a local network in order to send and receive files, and users can use clients written in Python, Java, and Go. These clients must use a standardized communication protocol that supports well-defined cryptographic primitives, and must guarantee confidentiality, integrity, and message authentication.

A key challenge in this environment is interoperability, as although each client is implemented in a different language and utilizes different libraries, they are still expected to discover one another, ensure identities are authenticated, and exchange files securely. In an effort to address these expectations, our system defines a shared wire protocol based on JSON messages and common cryptographic operations.

1.1 Project Objectives

When creating our secure P2P file-sharing application, we first needed to outline our primary objectives. These objectives were to ensure the application supported peer discovery on mDNS, mutual authentication of contacts, requesting of and sending files to and from other peers on the shared network, listing available files, and retrieving files from offline users. Regarding encryption, we also required confidentiality and the ability to migrate to a new key. Ensuring the integrity of all transferred files was also vital to prevent unauthorized tampering during transit. Beyond these core features, we prioritized the implementation of perfect forward secrecy, as it ensures that “the compromise of long-term keys does not allow an attacker to obtain past session keys.”[5] On the client side, we focused on secure local storage in an effort to protect

more sensitive data from being accessed even if the physical device was stolen. We also integrated comprehensive error handling to provide users with clear feedback regarding delivery failures or failed security checks, such as detected file tampering. We met these objectives by selecting libraries and security parameters that would aid us in meeting our security objectives, all while further maintaining a functional peer-to-peer environment.

2 SYSTEM OVERVIEW AND ARCHITECTURE

The architecture of our application is designed as a decentralized peer-to-peer system that uses mDNS to facilitate secure communication, and more importantly, secure file transfer between clients written in Python, Java, and Go. The system utilizes a multi-layered approach to peer interaction, and it begins with discovery via mDNS, prompting clients to broadcast their presence on a local network. Clients can automatically find available peers and mDNS reliably retrieves their name and IP/port. Once a peer is selected by one client, they can establish a TCP connection to begin communication. After this TCP connection is established, the two peers perform an authentication handshake. During this handshake, each side exchanges a long-term Ed25519 identity public key and a fresh X25519 ephemeral public key. The handshake transcript is signed with the Ed25519 private key and verified by the other side before communication continues. Using the exchanged X25519 ephemeral public keys, both peers compute a shared secret, which is then passed through HKDF-SHA256 to derive the symmetric session key used for the rest of the session. This session key is then used with AES-GCM to encrypt and authenticate all subsequent communication between the two peers. The presented identity key is then checked against the local trust store. If the key is already trusted, the session continues. If it is unknown, the user is shown the fingerprint and asked whether to trust it. If it conflicts with a previously trusted key, the connection is rejected. Following successful verification of the peer's identity, secure communication continues for file listing, requests, and retrieval of files from offline peers.

In regards to file retrieval and requesting files from an offline user, once a secure session has been established, a client can request a list of files available from another peer and then request a specific file for transfer. File metadata, which includes hashes and information of ownership, is used to verify integrity and track the original source of the selected file. In this way, the design supports retrieval from an offline user through cached file records. If a user who originally owned a file is no longer connected, another peer that had previously received and stored that file can provide it to the other user, as long as the cached metadata matches the original owner and the file hash verifies correctly. Thus, the system is able to support more flexible peer-to-peer file availability.

Files are also securely stored on the user’s device using AES-GCM and a key that is derived from the user’s input passphrase using PBKDF. Files that are received and placed in this encrypted store can be decrypted and written to the disk unencrypted to share with others.

The application also supports key migration to allow a user to continue secure file transfer after a public-private identity key is compromised. A user can generate a new public-private identity key and send a message to its peers with its new key, the new key signed with its old key, and the old key is signed with its new key, to prove that the user has access to both pairs. This does assume, however, that the user is able to change keys before the attacker does.

2.1 Protocol Design and Communication Flow Diagram

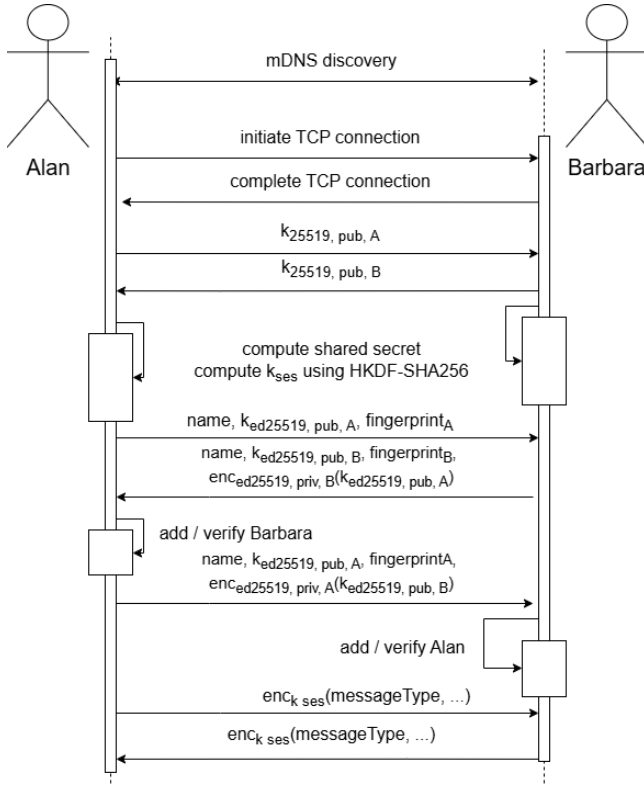


Figure 1: Diagram depicting the handshake protocol.

Messages are encoded as JSON and framed with a 4-byte length prefix so each client can reliably distinguish message boundaries on the stream.

As previously mentioned, a session key exchange is first performed before each user’s identity is exchanged. In our codebase, we only share the public key for each user back and forth in order to identify each user, but given more time we would modify our code bases to sign every peer’s fingerprints as shown, or more optimally, sign a challenge nonce to prove that a user has access to both their

private and public key. This extension would be implemented to prevent replay attacks.

2.2 Cryptography and Security Design

2.2.1 Key Exchange and Authentication. We made the decision to use Ed25519 for long-term identity keys and digital signatures. Ed25519 is a public-key signature scheme based on elliptic curve cryptography, and it is reliable as it is a secure and efficient method for authentication. It enables each client to maintain a persistent identity key pair, where the public key can be used to generate a fingerprint that identifies the peer. Thus, Ed25519 allows clients to verify that the other party is the expected peer.

Ephemeral key exchange for session keys is performed using X25519 elliptical curve Diffie-Hellman exchange because it allows two peers to securely establish a shared secret over an untrusted network without sending the secret itself. X25519 is well suited for this purpose because of its efficiency, it is widely used, and designed specifically for secure key agreement. Its usage also guarantees that each session generates a fresh shared secret, thus also improving security.

2.2.2 Key Derivation. Session keys are derived from the X25519 shared secret using HKDF-SHA256 because the raw shared secret produced by Diffie-Hellman is not intended to be used as an encryption key directly, and therefore HKDF provides a secure way to extract and expand that shared secret into a strong key that is suitable for symmetric encryption. Using SHA-256 within HKDF helps ensure that the derived session keys are secure against attacks. Ed25519 and X25519 use 256-bit keys, making them efficient while still providing strong protection against malicious intent.

2.2.3 File Message Format. When sending files back and forth between clients, they are sent along with the file owner’s name, the size of the file, the file’s SHA256 hash, a signature of the previously mentioned elements, and the owner’s public key. The signature of the vital elements is checked with the file owner’s public key to verify that they are unmodified. The hash is used to verify that the file is unmodified.

3 ERROR HANDLING AND EDGE CASES

Incoming files have their signature verified using the file owner’s public key and their hash checked to prevent in-transit modification. If these modifications are detected or other errors occur, then the user is notified and the files are not saved, but the clients continue their connection. If errors are raised when generating keys or when traversing directories, the clients exit as these errors impede the app usage.

3.1 Libraries

Across all clients in our project, the main external dependencies support cryptography and local peer discovery. The Python client makes use of cryptography[6] and zeroconf[7], the Java client utilizes Bouncy Castle[1], Gson[2], and JmDNS[4], and the Go client primarily uses the standard crypto library with libraries for PKCS#8 key handling[8] and mDNS peer discovery[3]. These libraries were chosen because they are able to provide very stable, well-supported

implementations of the primary features required for our P2P system. Cryptographic libraries securely handle identity keys, signatures, key exchange, and encryption without implementing algorithms from scratch. Our decision to rely on the cryptographic library is based around the following core principle when designing a secure system: “Never ever develop your own cryptographic algorithm unless you have a team of experienced cryptanalysts checking your design.”[5]

4 LIMITATIONS

There were several limitations encountered in the implementation of our P2P file-sharing application. One issue we identified was that the clients should send a nonce-based challenge when verifying to prove that the user being interacted with actually has the private key corresponding to the public key they present, but this is tested when sending files. This issue was discovered late into development and was not during protocol design, and so it is not addressed in the codebase. Another limitation is that trust on first use assumes there is an out of channel means of verifying that the user connecting is not malicious. File transfer also has a crucial limitation. File chunking was not fully working, so the system can only send files smaller than 10MB, which is the maximum message size supported by the protocol.

Another significant limitation was discovered in our key migration process. Due to the fact that a user’s key is changed by proving that they have access to their old key, an attacker that has knowledge of a user’s old private-public key pair can migrate to a new key themselves before the legitimate user is able to . This would be a key pair that the legitimate user does not know. A comparable analogy would be an attacker altering a user’s password on a website following their discovery of the user’s password. This would happen before the user is able to change the password themselves. This limitation is unavoidable given our design, as users only trust other users based on their public key fingerprints.

5 LESSONS LEARNED

With a project of this scope, we realized that working slowly and methodically was vital to success. Working in stages is instrumental when it comes to making certain that all components function properly. In order to make bugs easier to find, we should have made a minimum viable product by first ensuring all clients can communicate unencrypted and then verifying that they can authenticate properly with each other. We could have then moved on to making sure that all communication was encrypted, and finally adding all additional features after the previous steps were completed. Having checkpoints would have greatly reduced and distributed our workload more effectively over the course of the semester. Having regular testing at each step would also have helped improve the robustness of our clients.

We also learned that having a rigorous agreed upon protocol is the best foundation for a cryptographically secure peer-to-peer file sharing application. There were some aforementioned issues with our protocol, which we could have found if we had a more detailed planning stage that could have made use of diagrams, for example. Comparing these diagrams with those found in the class slides or the textbook would quickly have shown gaps in our security.

One final way we could have improved our collaboration would have been to initiate more frequent group coding and testing sessions. We had organized check-in meetings on a weekly basis, but working through different stages of the project together would have alerted us to shared issues earlier, giving us time to fix them quickly.

Our testing approach followed a similar, infrequent pattern and unfortunately would result in multiple bugs that needed to be resolved each meeting. Having more frequent testing meetings would have allowed us to test every step of the project individually, ensuring that we had stayed on track and avoided any major issues from piling up.

6 CONCLUSION

Our peer-to-peer file sharing application was instrumental in helping us learn how to tackle unexpected issues in our code, working as a team, and learning the basics of cryptography. We established a shared protocol based around stronger elliptic based versions of the methods for key derivation and signature functions in class, and were able to successfully transfer files to and from different clients. It was a necessary challenge that pushed us to our limits and taught us that oftentimes things do not go as planned. We also discovered some limitations and issues with our protocol which could be exploited by an attacker, which was to be expected. As the saying goes, never roll your own crypto.

REFERENCES

- [1] BouncyCastle. 2025. *Bouncy Castle – Open-source cryptographic APIs*. Retrieved April 6, 2026 from <https://www.bouncycastle.org/java.html>
- [2] Google. 2025. *Gson: A Java library that can be used to convert Java Objects into their JSON representation*. Retrieved April 6, 2026 from <https://github.com/google/gson>
- [3] grandcat. 2023. *zeroconf*. Retrieved April 6, 2026 from <https://github.com/grandcat/zeroconf>
- [4] JmDNS. 2025. *JmDNS*. Retrieved April 6, 2026 from <https://github.com/jmdns/jmdns>
- [5] Christof Paar and Güneysu Pelzl, Jan. 2024. *Understanding Cryptography: From Established Symmetric and Asymmetric Ciphers to Post-Quantum Algorithms* (2nd ed.). Springer, Berlin, Germany.
- [6] PyCa. 2026. *Welcome to pyca/cryptography*. Retrieved April 6, 2026 from <https://cryptography.io/en/latest/>
- [7] python zeroconf. 2025. *python-zeroconf*. Retrieved April 6, 2026 from <https://github.com/python-zeroconf/python-zeroconf>
- [8] youmark. 2024. *pkcs8*. Retrieved April 6, 2026 from <https://github.com/youmark/pkcs8>